



The Department of Electronics And Communication Course Level Problem Based Learning

Course Title: DDHDL Verilog

Course Code:22ECE42

Semester: Fourth Semester

Project Report Titled

FPGA implementation of ECG signals using FIR filter

Submitted By

Hemanth	4SO24EC042
Ethan Ranger	4SO24EC033
Dhathri P G	4SO24EC030
Deeksha K	4SO24EC029

Under the Guidance of P.C Vijay Ganesh

(Assistant Professor , B.E , M. Tech, MIEEE)

PBL Secondary Research Report

(FPGA-Based Design Project)

Chapter 1: Introduction

1.1 Background: In modern healthcare and environmental monitoring, continuous data acquisition from sensors such as ECG electrodes and gas detectors plays a crucial role in analyzing physiological conditions and surrounding air quality. These signals are often corrupted by various types of noise, including powerline interference, motion artifacts, and high-frequency disturbances, making digital filtering an essential step before further analysis. Traditionally, such filtering operations are performed using software-based implementations on microcontrollers or digital signal processors. However, these systems execute instructions sequentially, which introduces processing delays and increases power consumption, especially when handling high sampling rates or multi-channel data. This becomes a major limitation in real-time applications where immediate response and energy efficiency are critical.

With the rapid growth of wearable technology and IoT-based monitoring systems, there is a strong need for faster and more efficient signal processing solutions. Field Programmable Gate Arrays (FPGAs) offer a powerful alternative by enabling hardware-level parallelism, pipelining, and reconfigurability. Unlike software approaches, FPGAs can process multiple data samples simultaneously, significantly reducing latency and improving throughput. Additionally, FPGA-based systems can be optimized for specific applications, leading to better performance and lower power usage. These advantages make FPGAs highly suitable for implementing real-time FIR filters in biomedical and environmental monitoring systems, ensuring accurate and reliable signal processing.

1.2 Problem Statement:

Real-time biomedical and environmental monitoring systems require efficient digital signal processing. Software-based FIR filtering can introduce latency and increased power consumption in wearable and IoT devices. Design and implement an FPGA-based hardware system capable of performing real-time digital filtering for streaming input data..

1.3 Objectives:

The primary objective of this project is to develop an efficient FPGA-based digital filtering system capable of processing streaming input signals in real time. The design aims to overcome the limitations of conventional software-based filtering by leveraging parallel processing and pipelined architecture to achieve high-speed operation with minimal delay. A key goal is to implement a robust FIR filter structure using essential hardware components such as shift registers for storing past samples, coefficient ROM for predefined filter weights, multipliers for performing sample-coefficient operations, and adders for accumulating results.

Another important objective is to minimize power consumption while maintaining high performance, making the system suitable for portable and battery-operated devices like wearable health monitors and remote sensing units. The project also focuses on achieving efficient resource utilization within the FPGA, ensuring that logic elements, flip-flops, and DSP blocks are optimally used. Furthermore, the design is intended to be scalable and flexible, allowing the implementation of different types of filters such as low-pass, high-pass, and notch filters based on application requirements. Ultimately, the system aims to provide accurate noise removal, improved signal quality, and reliable real-time operation for both biomedical and environmental monitoring applications.

.

Chapter 2: Literature Review (Secondary Research)

2.1 Review of Existing Work:

Architecture Used:

- Pipelined FIR Architecture (parallel MAC structure)
- Uses shift registers (Z^{-1} delays) + multiplier–adder tree
- Optimized with CSD/Distributed Arithmetic multipliers

Throughput Achieved:

- High throughput due to parallel processing
- One output sample per clock cycle (after pipeline fill)
- Suitable for real-time ECG signals

Resource Utilization:

- LUTs: Used for control logic, adders, and DA implementation
- Flip-Flops (FFs): Used in pipeline registers and shift registers
- DSP slices: Reduced usage (due to CSD/DA instead of full multipliers)

Limitations:

- Fixed coefficients (no adaptive filtering)
- No input buffering (timing mismatch possible)
- Moderate area usage due to multiple registers
- Power optimization not considered
- Limited noise/artifact handling for real ECG signals

2.2 Comparative Analysis:

paper	Architecture used	Throughput (MSPS)	Area	Limitations
Window-based FIR	MATLAB (Simulation)	Not Reported	N/A	No hardware implementation
CSD Pipelined FIR	Kintex-7 + ASIC (180nm)	High (Pipelined, ~100-300 MSPS)	High (Reduced LUTs)	Fixed coefficients
Distributed Arithmetic (DA) FIR	Altera Cyclone FPGA	High (~100-230 MSPS)	Very High	Large LUT size
Partly Serial Equi-ripple FIR	Virtex-5 FPGA	Good (~100-200 MSPS)	Good	Lower than fully parallel
LMS Adaptive FIR	Spartan-3 FPGA	~3.9 MSPS (real-time)	Not Reported	Lower than fully parallel
Multilevel Cascade FIR	Virtex-6 FPG	High (~150-300 MSPS)	Moderate	Higher latency
Quantum-inspired FIR	Artix-7 FPGA	High (~150-250 MSPS)	52.07%	Quantization distortion
FIR + DWT + SVM	Spartan-6 FPGA	Moderate (~80-150 MSPS)	Not Reported	High complexity

2.3 Research Gap Identification:

Existing ECG filtering systems based on microcontrollers suffer from high latency, limited processing speed, and higher power consumption, making them unsuitable for real-time and wearable applications. Additionally, they lack efficient multi-stage filtering and parallel processing capabilities. This project addresses these limitations by implementing a high-speed, low-power FPGA-based pipelined FIR filtering system with

optimized hardware architecture and enhanced noise removal performance.”

Chapter 3: Proposed System Design

3.1 Design Justification:

1. FPGA-Based High Performance:

- Replaces MCU-based FIR (high latency & power)
- FPGA enables parallel processing
- Achieves high speed + low power
- Suitable for real-time ECG (wearables/IoT)

2. Multi-Stage ECG Filtering:

- Uses LPF + Notch + HPF cascade
 - Removes:
 - High-frequency noise (LPF)
 - Powerline interference (Notch)
 - Baseline drift (HPF)
 - Produces clean and accurate ECG signal

3. Pipelined Architecture:

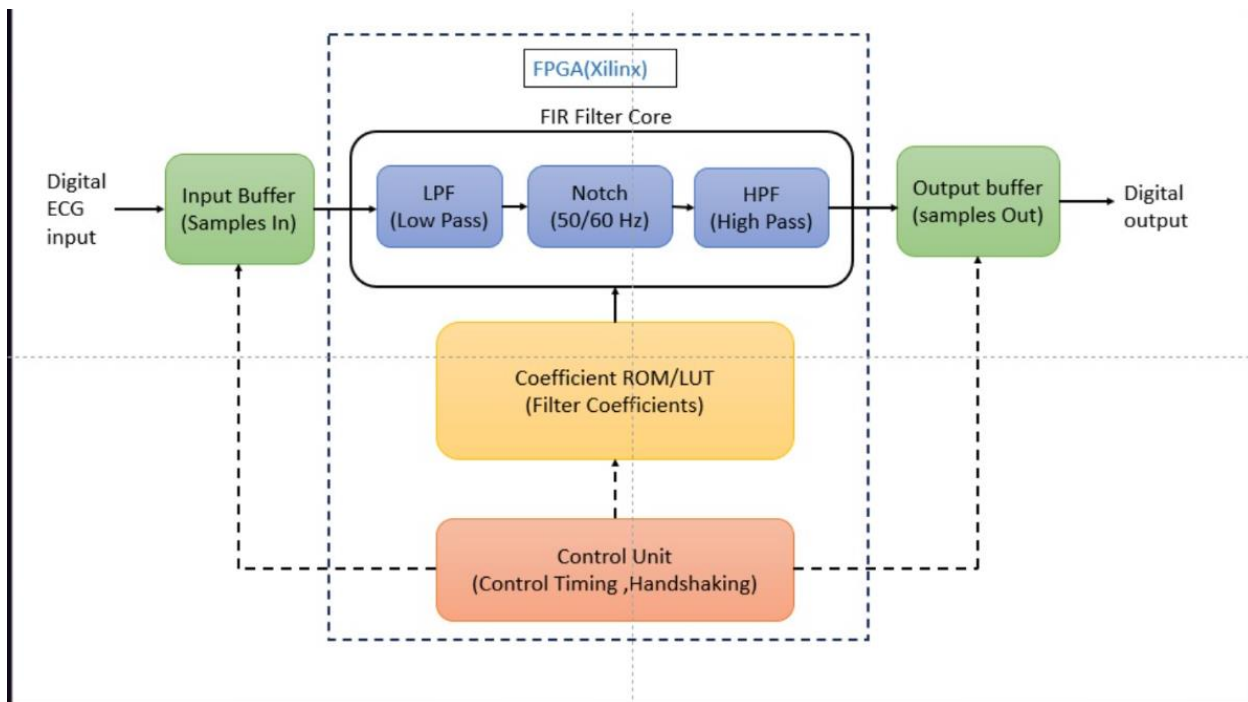
- Uses Z^{-1} delay elements
 - Reduces critical path delay
 - Enables continuous high-speed processing
 - Ensures low latency & real-time response
- #### 4. Optimized Hardware Usage
- Uses efficient multipliers (CSD/DA)
 - Implements CSA adder tree
 - Reduces:
 - Area

- Power consumption
- Switching activity

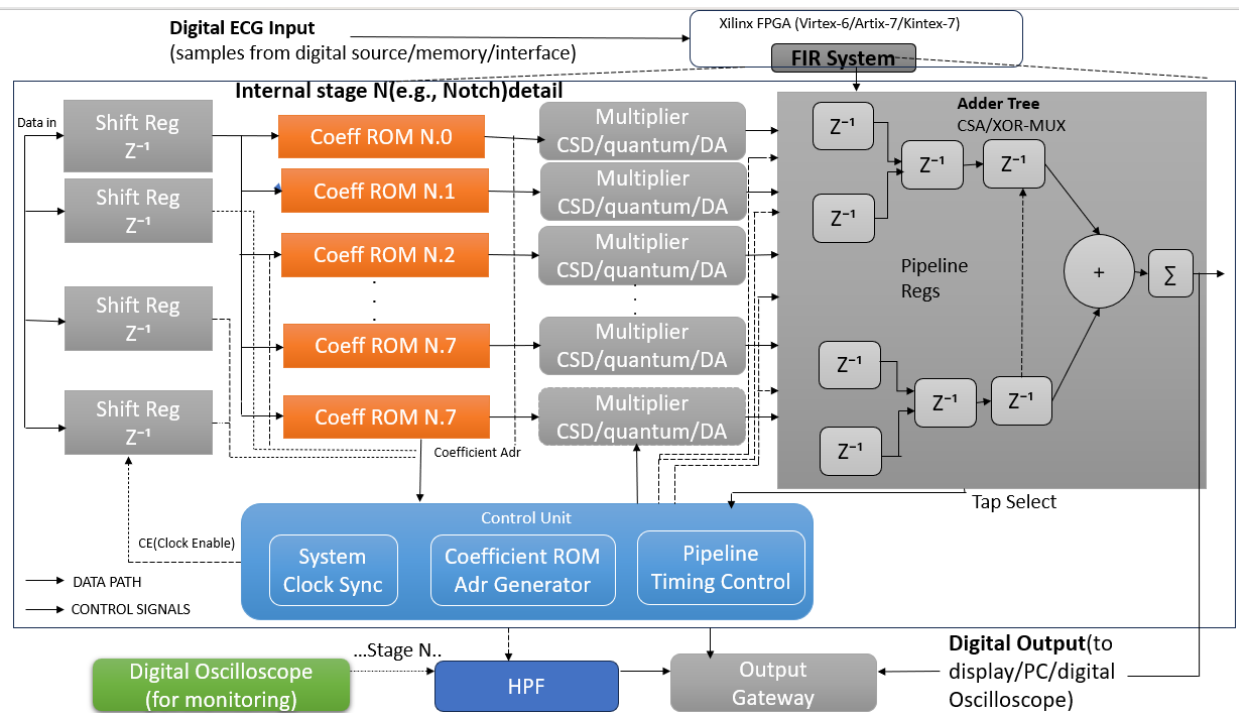
5. Flexible System Control:

- Includes buffers + control unit
- Ensures synchronization & reliability
- Uses Coefficient ROM/LUT
- Allows adaptation to different signals/patients

3.2 Block Diagram:



3.3 Architecture Diagram:



Chapter 4: Leaf Cell Design:

4.1 Identification of Leaf Cells

- Shift Register Block (Z^{-1} delays)
- Coefficient ROM
- Multiplier Block
- Adder Tree (CSA / XOR-MUX)
- Pipeline Registers (Z^{-1} inside adder tree)

4.2 Leaf Cell Description

1. Shift Register Block (Z^{-1} delays):

Leaf cells:

- D Flip-Flops (D-FFs)
- Clock buffers
- Enable logic (AND gates)

D Flip-Flop (D-FF):

A D flip-flop is a basic storage element that stores one bit of data and updates its output only on a clock edge. It is widely used in shift registers, pipeline stages, and synchronization circuits in FPGA designs.

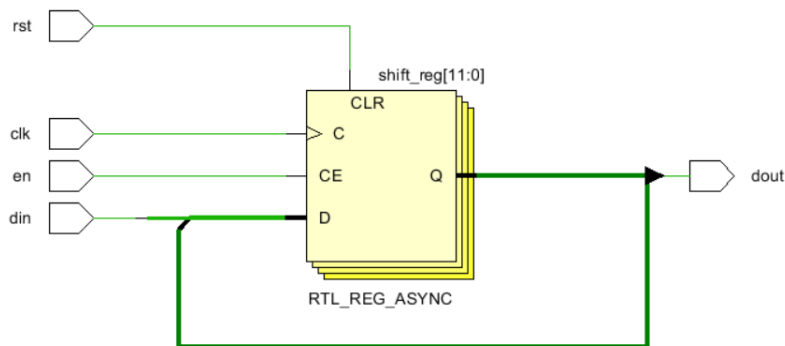
Clock Buffer:

A clock buffer distributes the clock signal evenly across the circuit. It ensures proper timing and reduces signal delay and distortion.

Enable logic (AND gates):

Enable logic controls whether data should be processed or held. AND gates are commonly used to allow signal flow only when an enable signal is active, helping in controlled data shifting and power saving.

RTL Design:



A 12-bit shift register stores and shifts data one bit at a time on each clock cycle. When the reset ('rst') is asserted, all bits in the register are cleared to zero. After reset, on every rising edge of the clock ('clk'), if the enable ('en') signal is high, the data inside the register shifts one position, and the new input bit ('din') enters at one end of the register. Each existing bit moves to the next position, and the oldest bit eventually reaches the most significant position, which is connected to the output ('dout'). As a result, the output represents the input data delayed by 12 clock cycles. If the enable signal is low, the shifting stops and the register holds its current value.

2. Coefficient ROM

Leaf cells:

- Memory cells (SRAM/ROM cells)
- Address decoder
- Word line & bit line drivers
- Sense amplifiers

Memory cells (SRAM/ROM cells):

Memory cells store binary data such as filter coefficients. ROM cells hold fixed values, while SRAM cells allow read/write operations. These cells are the fundamental storage units in digital systems.

Address decoder:

The address decoder selects a specific memory location based on input address lines. It activates one word line corresponding to the given address, allowing access to stored data.

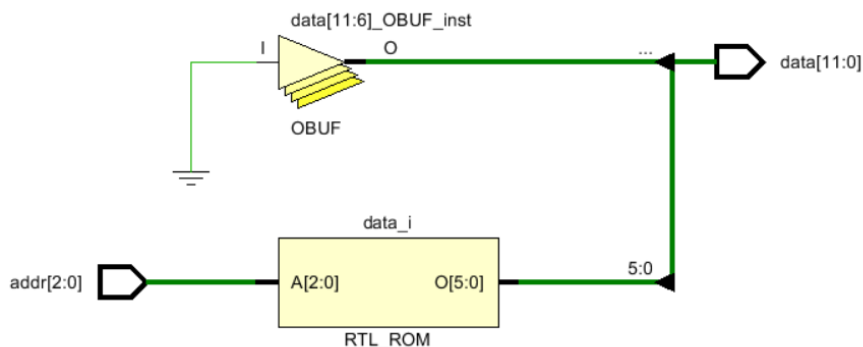
Word line & bit line drivers:

These drivers control access to memory cells. Word line drivers activate rows, while bit line drivers carry data to and from the cells. They ensure fast and reliable memory operation.

Sense amplifiers:

Sense amplifiers detect small voltage differences in memory cells and amplify them to readable logic levels. They are crucial for accurate and fast data retrieval from memory.

RTL Design:



A ROM stores fixed data and outputs it based on the input address. In your design, a 3-bit address selects one of 8 locations, and the ROM provides a predefined 6-bit output. This data is then sent to the output through a buffer.

3. Multiplier Block

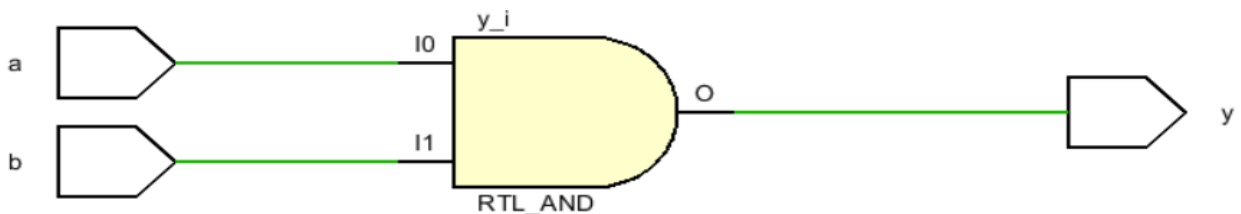
Leaf cells:

- AND gates (partial product generation)
- Adders (Half Adder, Full Adder)
- Booth encoder
- Shifter

AND gates:

In multipliers, AND gates generate partial products by multiplying individual bits of operands. These partial products are later summed to produce the final result.

RTL Design:



This circuit represents a basic AND gate. It takes two inputs, a and b, and produces an output y. The output is 1 only when both inputs are 1; otherwise, the output is 0. In simple terms, it performs logical multiplication:

$$* a = 0, b = 0 \rightarrow y = 0$$

$$* a = 0, b = 1 \rightarrow y = 0$$

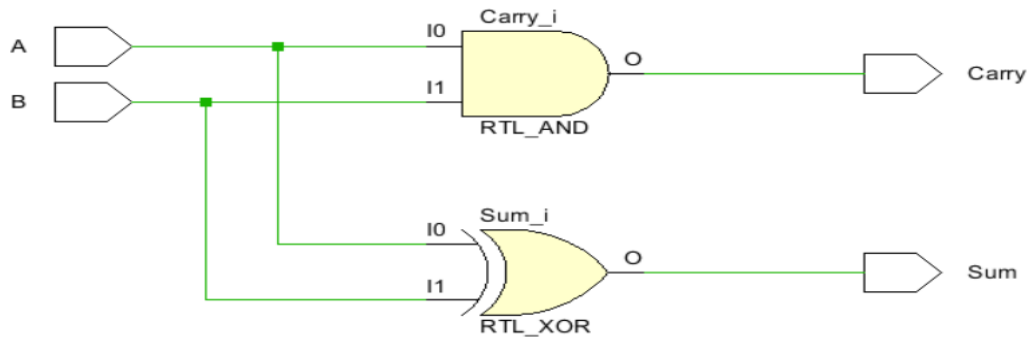
$$* a = 1, b = 0 \rightarrow y = 0$$

* $a = 1, b = 1 \rightarrow y = 1$ Adders:

- **Half Adders (HA):**

A half adder adds two binary bits and produces a sum and carry output. It is a basic arithmetic unit used in building larger adders.

RTL Design:

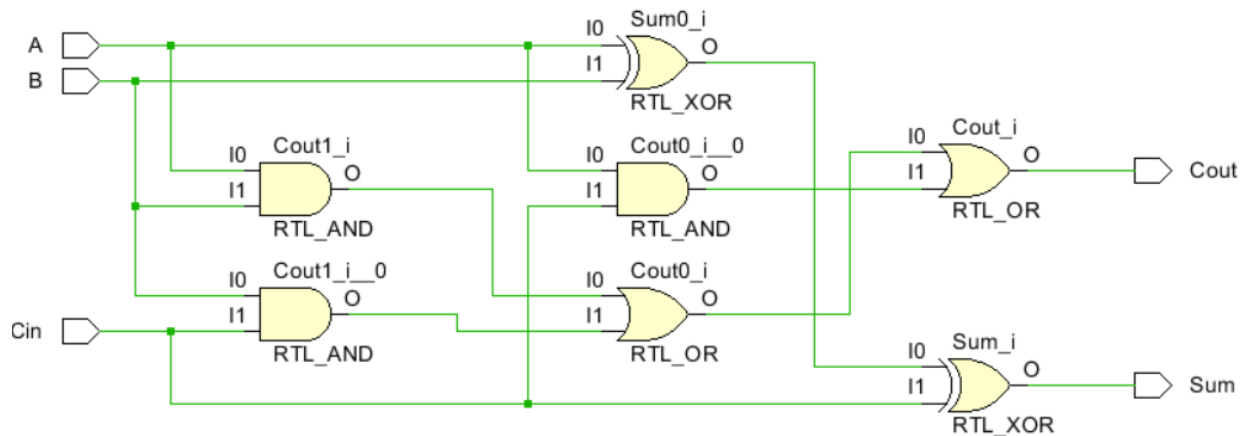


A half adder is a digital circuit that adds two single-bit inputs (A and B) and produces two outputs: Sum and Carry. The Sum is generated using an XOR operation ($A \oplus B$), which gives 1 when the inputs are different. The Carry is generated using an AND operation ($A \cdot B$), which gives 1 only when both inputs are 1. Thus, it performs basic binary addition without considering any carry input from a previous stage.

- **Full Adders (FA):**

A full adder adds three inputs (two bits and a carry-in) and produces a sum and carry-out. It is widely used in arithmetic circuits like multipliers and adder trees.

RTL Design:

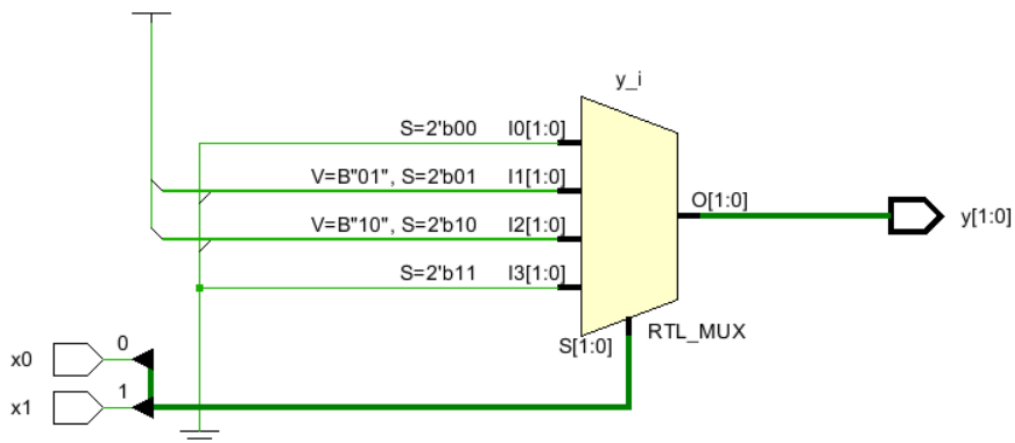


A full adder adds three 1-bit inputs (A, B, and Cin) and produces Sum and Carry-out (Cout). The Sum is the XOR of all three inputs, and Cout is 1 when at least two inputs are 1.

Booth Encoder:

The Booth encoder is used in optimized multipliers to reduce the number of partial products. It improves speed and reduces hardware complexity by encoding multiplier bits efficiently.

RTL Design:



A Booth multiplier is used for fast multiplication of signed binary numbers. It works by checking two bits of the multiplier at a time (current and previous bit) to decide whether to add, subtract, or skip the multiplicand. This reduces the number of operations compared to normal multiplication, making it more efficient, especially for numbers with consecutive 1s.

Shifters:

Shifters move binary data left or right. In multiplication, shifting aligns partial products according to their bit positions before addition.

4. Adder Tree (CSA / XOR-MUX)

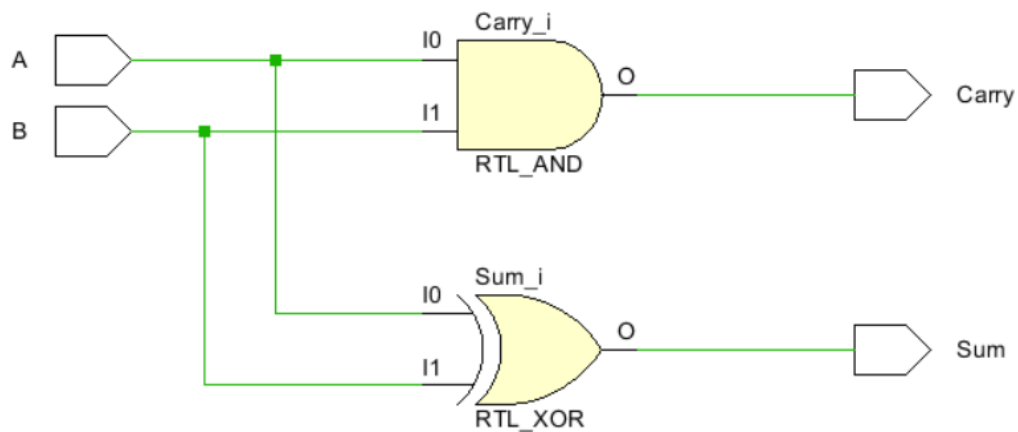
Leaf cells:

- Full Adders (FA)
- Half Adders (HA)
- XOR gates
- Multiplexers (MUX)
- Carry Save Adders (CSA units)

Half Adders (HA):

A half adder adds two binary bits and produces a sum and carry output. It is a basic arithmetic unit used in building larger adders.

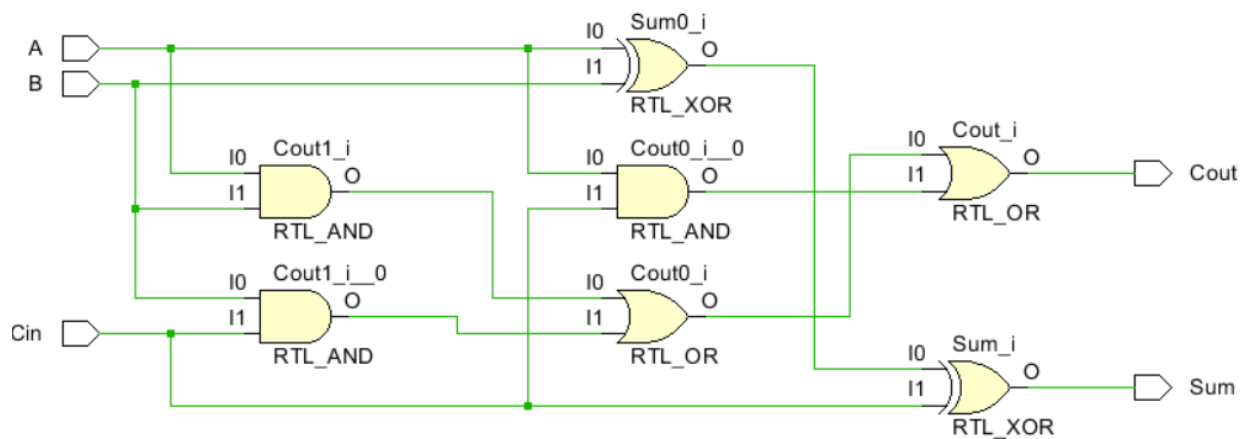
RTL Design:



Full Adders (FA):

A full adder adds three inputs (two bits and a carry-in) and produces a sum and carry-out. It is widely used in arithmetic circuits like multipliers and adder trees.

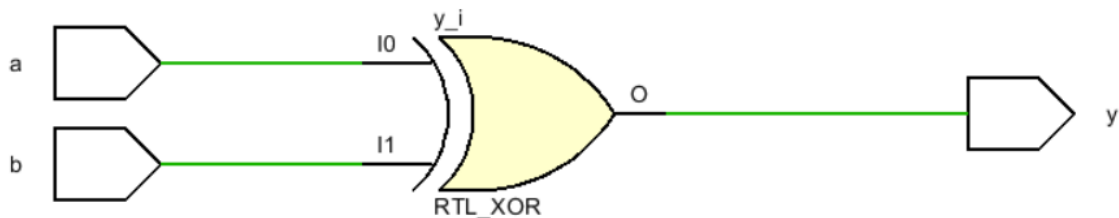
RTL Design:



XOR Gates:

XOR gates are used for sum calculation in adders and for logic comparison. They output high only when inputs are different, making them essential in arithmetic and control logic.

RTL Design:

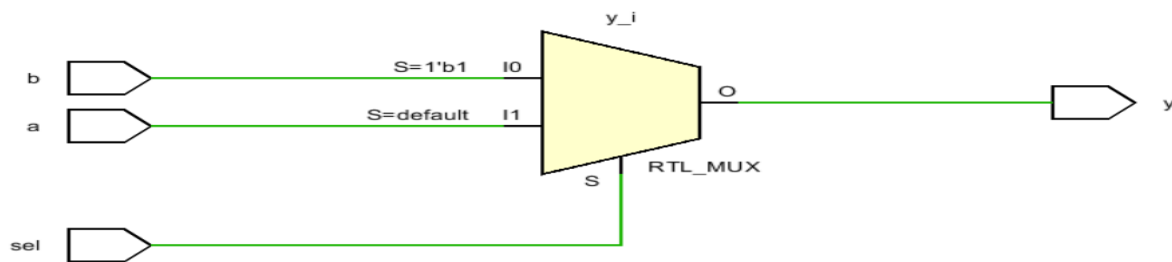


an XOR (Exclusive OR) gate. It takes two inputs, a and b, and produces an output y that is 1 only when the inputs are different. If both inputs are the same (both 0 or both 1), the output is 0. In simple terms, it performs addition without carry in binary.

Multiplexers (MUX):

Multiplexers select one input from multiple inputs based on control signals. They are used for data routing, tap selection, and output selection in digital systems.

RTL Design:

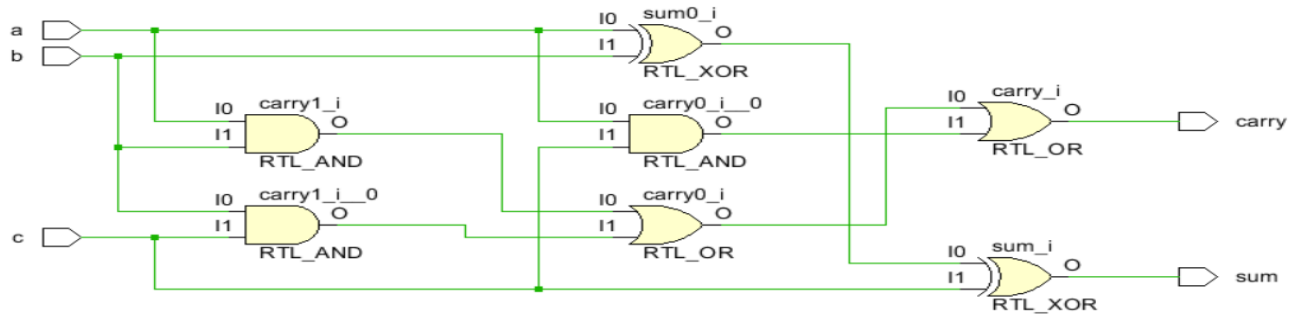


a 2-to-1 Multiplexer (MUX). It selects one of the two inputs (a or b) based on the select signal sel. If sel = 0, the output y is a; if sel = 1, the output y is b.

Carry Save Adders (CSA):

CSAs are used to add multiple binary numbers without immediately propagating carry. This improves speed in multi-operand addition, especially in adder trees of FIR filters.

RTL Design:



A Carry Save Adder (CSA) is used to add three binary numbers (a, b, c) simultaneously without immediately propagating the carry. Instead of waiting for carries to ripple through like in a normal adder, it produces two outputs: sum and carry. The sum is calculated using XOR operations, and the carry is generated using AND/OR logic. The carry is “saved” and passed to the next stage rather than being added right away. This makes CSA much faster, especially in multipliers, because it reduces delay caused by carry propagation.

5. Pipeline Registers (Z^{-1} inside adder tree)

Leaf cells:

- D Flip-Flops
- Clock gating cells

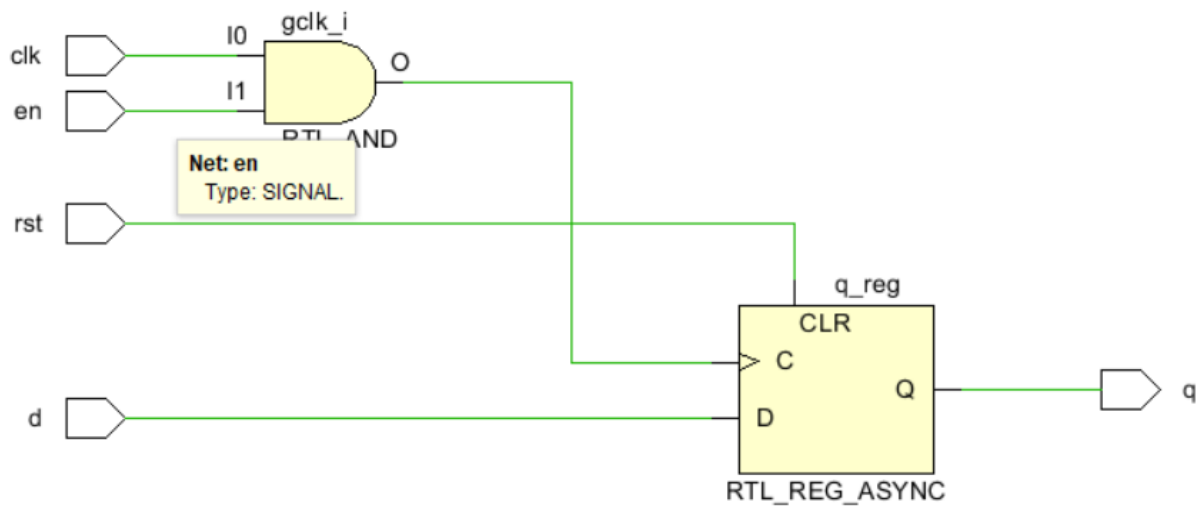
D Flip-Flop (D-FF):

A D flip-flop is a basic storage element that stores one bit of data and updates its output only on a clock edge. It is widely used in shift registers, pipeline stages, and synchronization circuits in FPGA designs.

Clock Gating Cells:

Clock gating cells reduce power consumption by disabling the clock signal to inactive parts of the circuit. This prevents unnecessary switching activity.

RTL Design:



The clock (**clk**) is ANDed with the enable signal (**en**) to generate a gated clock (**gclk**). The flip-flop updates its output **q** only when **en** = 1, because only then the gated clock allows clock edges to pass. When **en** = 0, no clock reaches the flip-flop, so it holds its previous value. The reset (**rst**) asynchronously clears the output to 0.

Leaf cell simulation:

1. Shift register:

Simulation:

#10 **rst** = 0; **en** = 1;

din = 1; #10;

din = 0; #10;

din = 1; #10;

din = 1; #10;

din = 0; #10;

din = 1; #10;

din = 1; #10;

din = 0; #10;

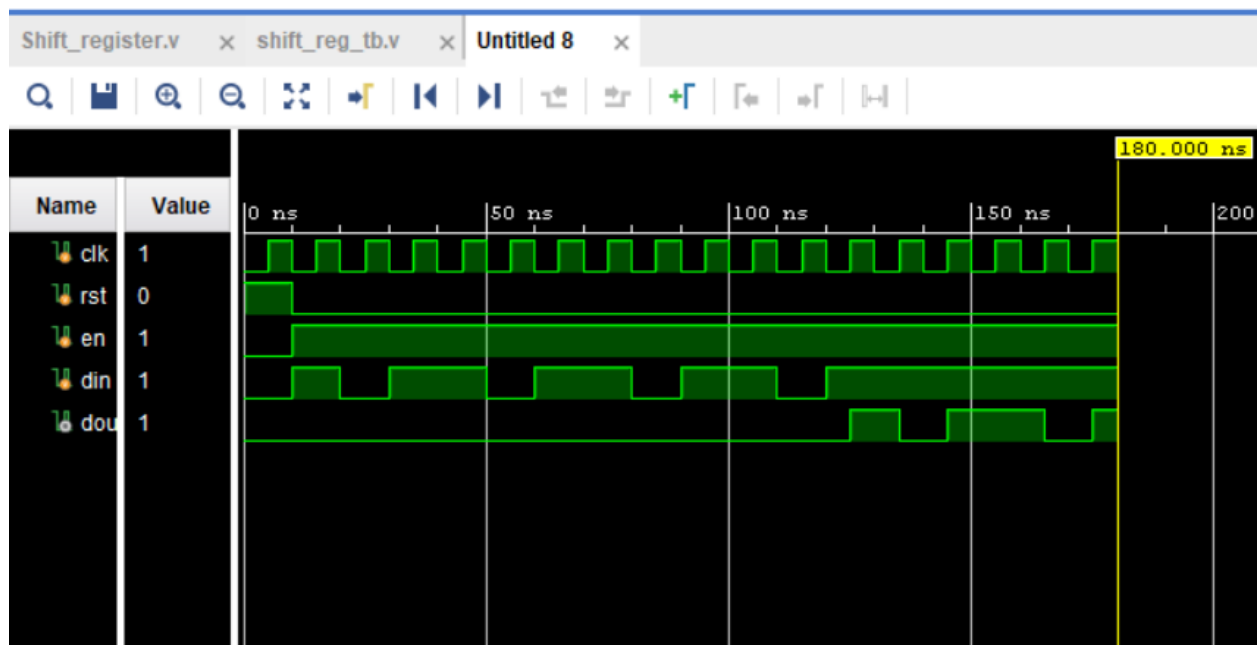
din = 1; #10;

din = 1; #10;

din = 0; #10;

din = 1; #10;

Waveform:



2.Coefficient ROM:

Simulation:

```
module coeff_rom_tb ();
```

```
reg [2:0] addr;
```

```

wire [11:0] data;

coeff_rom uut (. addr (addr), .data(data));

integer i;

initial begin

$monitor ("addr=%b data=%d", addr, data);

For (i=0; i<8; i=i+1) begin

addr = i;

#10;

end

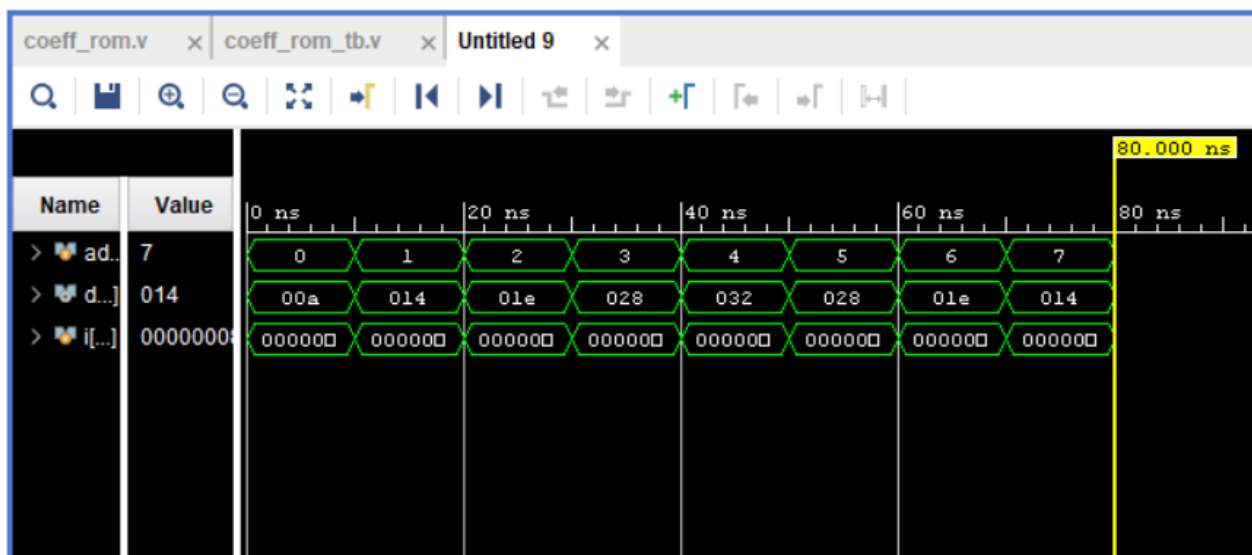
$finish;

end

endmodule

```

Waveform:



AND gate:

Simulation:

a=0; b=0;

a=0; b=1;

a=1; b=0;

a=1; b=1;

Waveform:



3.Multiplier block

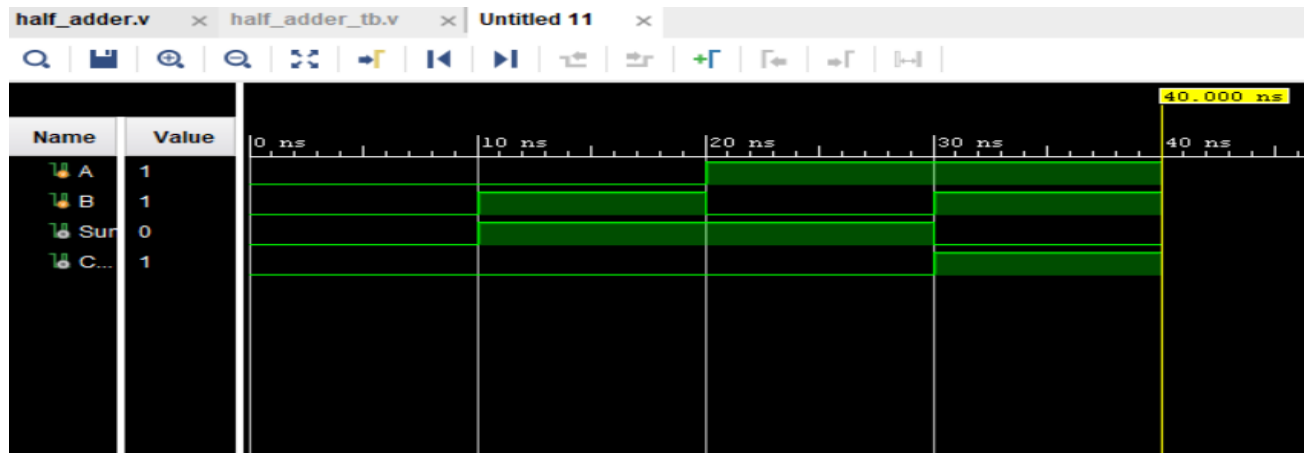
Half Adder

Simulation:

#10; A=0; B=0; #10; A=0; B=1;

#10; A=1; B=0; #10; A=1; B=1;

Waveform:



Full Adder:

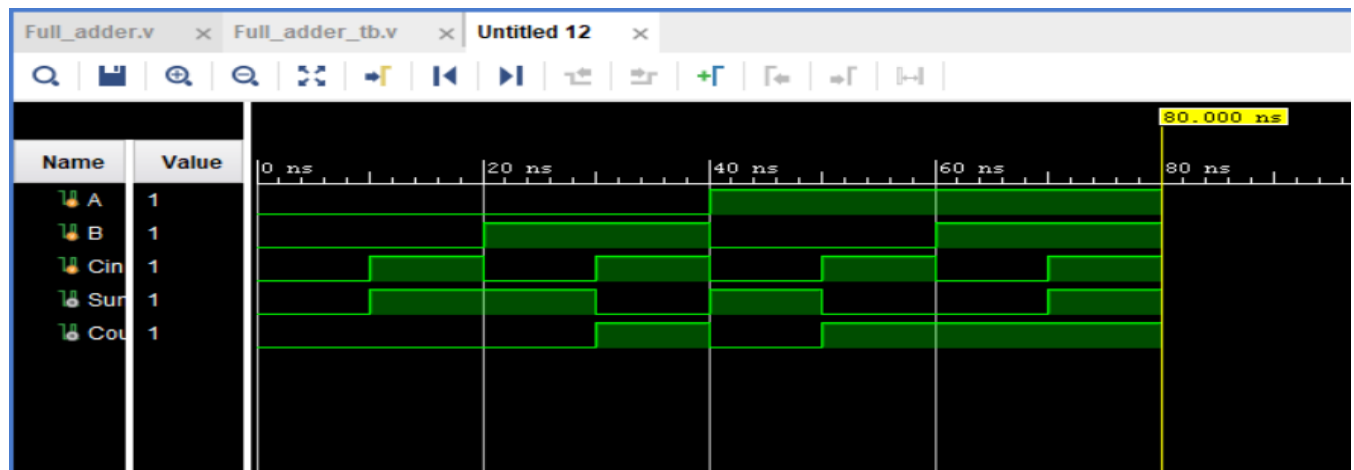
Simulation:

```

A=0; B=0; Cin=0; #10;
A=0; B=0; Cin=1; #10;
A=0; B=1; Cin=0; #10;
A=0; B=1; Cin=1; #10;
A=1; B=0; Cin=0; #10;
A=1; B=0; Cin=1; #10;
A=1; B=1; Cin=0; #10;
A=1; B=1; Cin=1; #10;

```

Waveform:

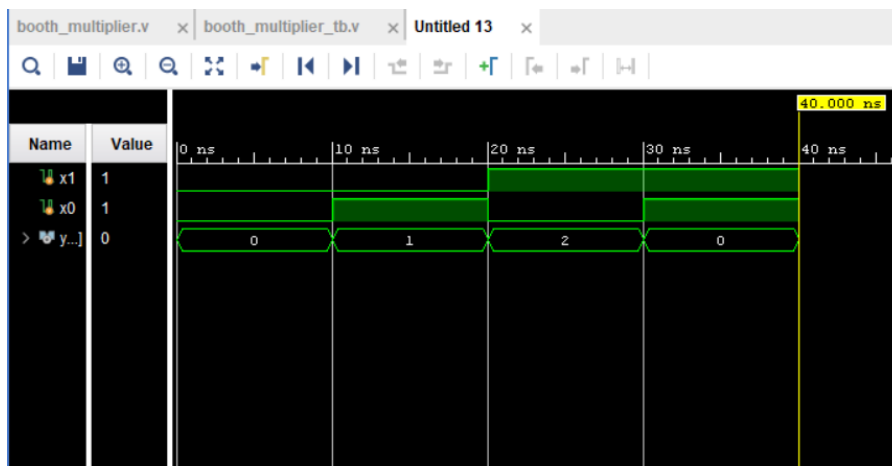


Booth encoder:

Simulation:

```
x1=0; x0=0;  
#10 x1=0; x0=1;  
#10 x1=1; x0=0;  
#10 x1=1; x0=1;
```

Waveform:



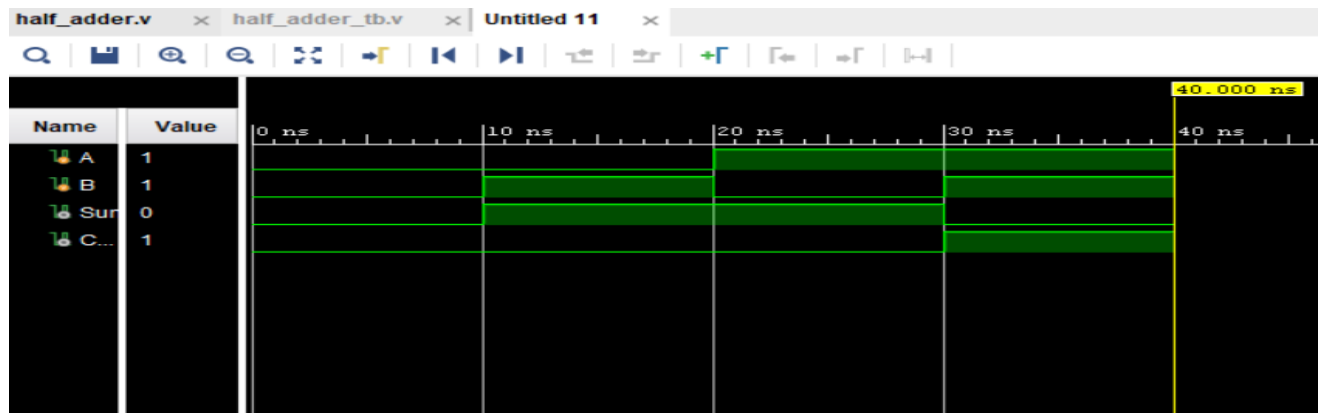
4.Adder tree

Half adder

Simulation:

```
#10; A=0; B=0; #10; A=0; B=1;  
  
#10; A=1; B=0; #10; A=1; B=1;
```

Waveform:



Full adder:

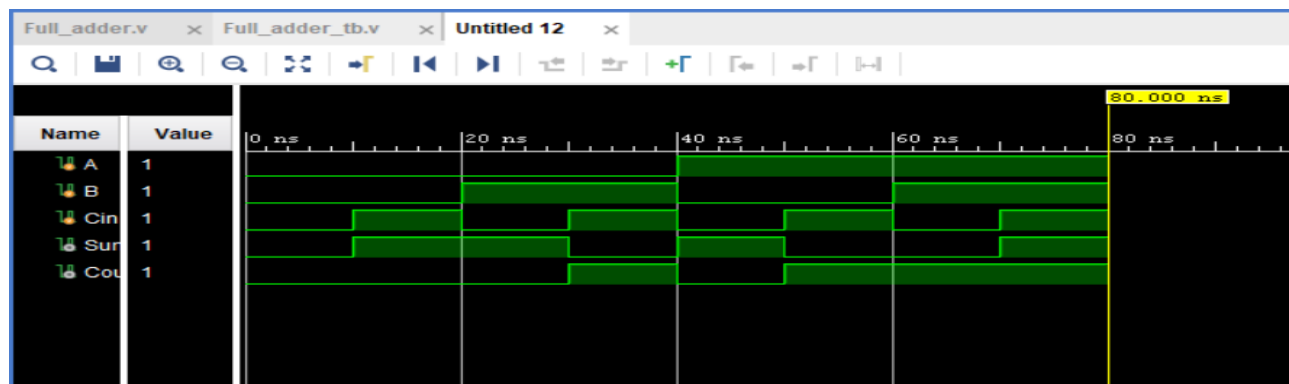
Simulation:

```

A=0; B=0; Cin=0; #10;
A=0; B=0; Cin=1; #10;
A=0; B=1; Cin=0; #10;
A=0; B=1; Cin=1; #10;
A=1; B=0; Cin=0; #10;
A=1; B=0; Cin=1; #10;
A=1; B=1; Cin=0; #10;
A=1; B=1; Cin=1; #10;

```

Waveform:



Xor gate

Simulation:

```

a=0; b=0;
#10 a=0; b=1;
#10 a=1; b=0;

```

#10 a=1; b=1;

Waveform:



CSA Adder

Simulation:

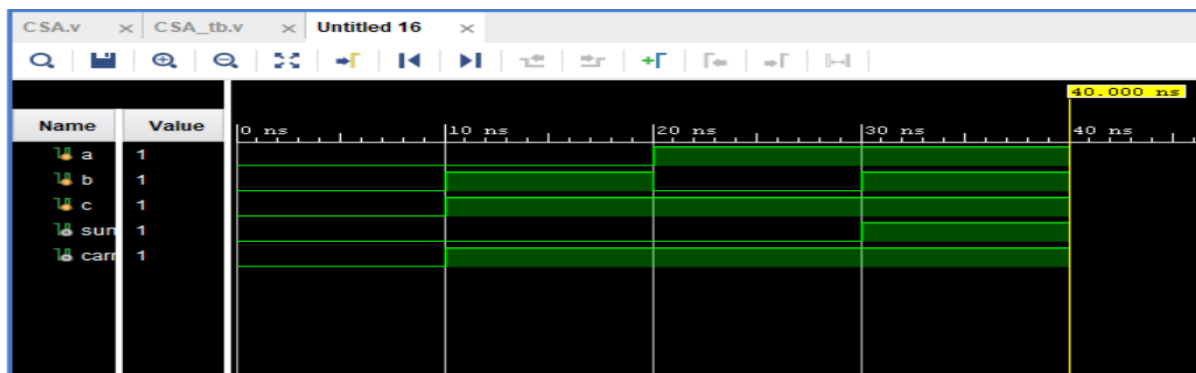
a=0; b=0; c=0;

a=0; b=1; c=1;

a=1; b=0; c=1;

a=1; b=1; c=1

Waveform:



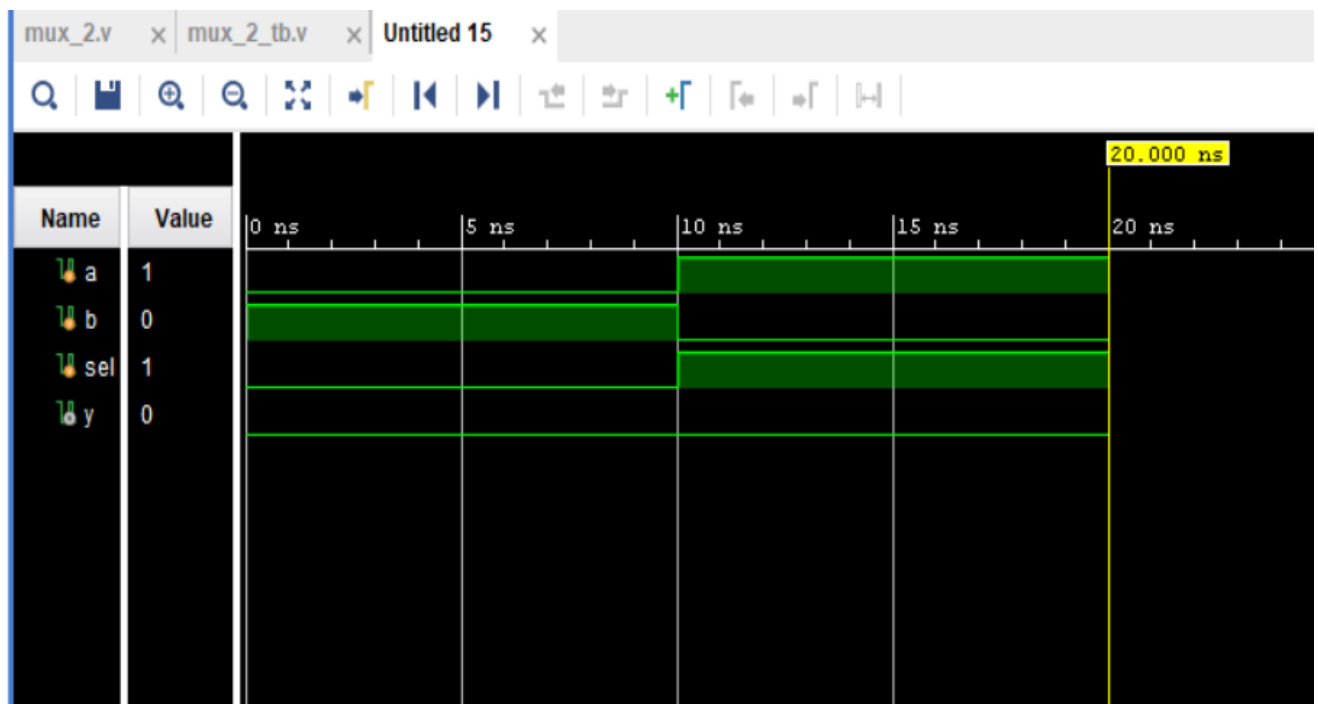
MUX

Simulation:

a=0; b=1; sel=0;

a=1; b=0; sel=1;

Waveform:



5.Pipeline register

D FlipFlop:

Simulation:

clk = 0; rst = 1; en = 0; d = 0;

#10 rst = 0; en = 1;

d = 1;

#10;

d = 0;

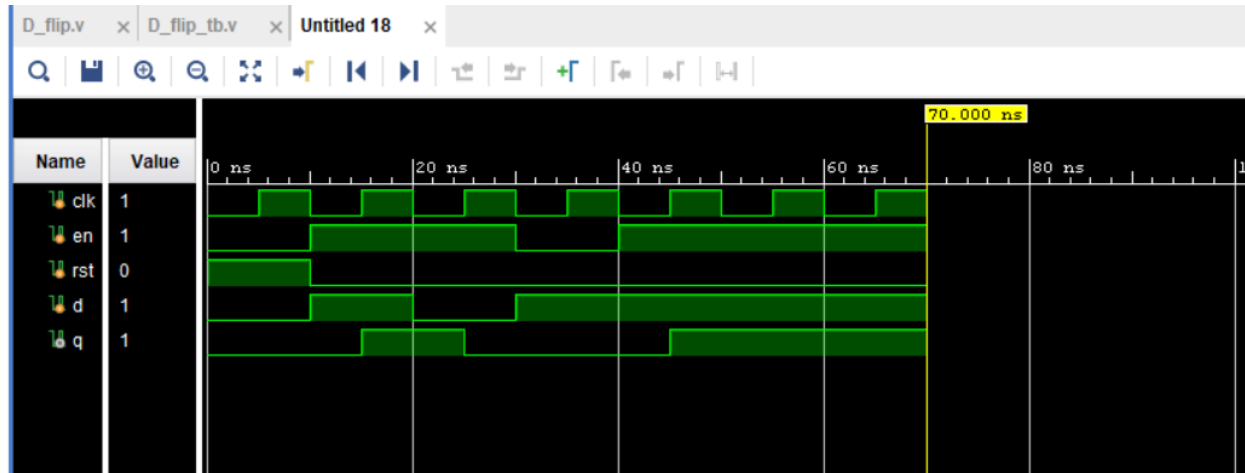
#10;

```

en = 0;
d = 1;
#10;
en = 1;
d = 1;
#10;

```

Waveform:



References:

IEEE papers:

References:

- [1] P. C. Bhaskar and M. D. Uplane, "FPGA Based Digital FIR Multilevel Filtering For ECG Denoising," in Proc. 2015 International Conference on Information Processing (ICIP), Vishwakarma Institute of Technology, Pune, India, Dec. 16–19, 2015, pp. 733–738.
- [2] M. Prabhavathy and S. M. Sakthivel, "VLSI Implementation of CSD Based Efficient FIR Filter for ECG Signal Denoising," in Proc. 2024 IEEE 4th International Conference on VLSI Systems, Architecture, Technology and Applications (VLSI SATA), 2024, doi: 10.1109/VLSISATA61709.2024.10560274.
- [3] B. Sriram, A. V. Thanvi, B. V. V. Reddy, and V. B. K. L. Aruna, "Strategic Reduction of Area and Power in FIR Filter Architecture for ECG

Signal Acquisition," in Proc. 2025 14th International Conference on System Modeling & Advancement in Research Trends (SMART), Teerthanker Mahaveer University, Moradabad, India, Nov. 14–15, 2025, pp. 574–578, doi: 10.1109/SMART66937.2025.11389312.

[4] R. Rasu, P. Shanmuga Sundaram, and N. Santhiyakumari, "FPGA Based Non-Invasive Heart Rate Monitoring System for Detecting Abnormalities in Fetal," in Proc. 2015 International Conference on Space Science and Communication (SPACES), K L University, 2015, pp. 375–379.

[5] L. Nian-qiang, H. Si-Yu, and C. Shi-Yao, "Application of Distributed FIR Filter Based on FPGA in the Analyzing of ECG Signal," in Proc. 2010 International Conference on Intelligent System Design and Engineering Application (ISDEA), 2010, pp. 335–338, doi: 10.1109/ISDEA.2010.265.

[6] B. Singh, R. Mehra, and Chandni, "Reconfigurable FIR Filter for Denoising of ECG Signal," in Proc. 2016 IEEE International Conference, NITTTR Chandigarh, India, 2016.

[7] S. Sumathi, A. Balaji Ganesh, S. Umakumari, H. Priyadharshini, and S. Srimathi, "Arrhythmia Detection and ECG Signal Analysis using Digital Filtering Techniques," in Proc. 7th International Conference on Communication and Electronics Systems (ICCES 2022), 2022, pp. 6–11, doi: 10.1109/ICCES54183.2022.9835793.

[8] T. Singh, P. Agarwal, and V. K. Pandey, "ECG Baseline Noise Removal Techniques Using Window Based FIR Filters," in Proc. 2014 International Conference on Medical Imaging, m-Health and Emerging Communication Systems (MedCom), Greater Noida, India, 2014, pp. 131–136.

Appendix

A.1 Tools Used:

Matlab , Vivado

A.2 Additional Supporting Data:

1. Simulation Results*

- Verified using HDL simulation (Vivado)
- Waveforms show:
 - Noisy ECG input
 - High-Pass Filter output
 - Low-Pass Filter output
 - Final filtered ECG (clean signal)

2. Signal Improvement:

- Clear reduction in noise and baseline drift
- Improved ECG waveform clarity
- Accurate preservation of cardiac features

3. Performance Metrics:

- Low latency due to pipelined architecture
- High processing speed through parallel execution
- Reduced power consumption compared to MCU-based systems

4. Hardware Efficiency:

- Optimized resource usage (LUTs, FFs)
- CSA-based adder tree reduces computation delay
- Efficient multiplier usage minimizes hardware overhead

5. System Reliability:

- Stable and continuous data processing
- Proper synchronization using buffers and control logic
- Suitable for real-time applications

6. Application Scope:

- Wearable ECG monitoring devices
- Remote health monitoring systems
- Real-time biomedical signal processing